GLUD Fried Chicken (GFC)

Taller de Métricas de Software

Diseño de Software

Nombre	Código
Daniel Mateo Ballesteros Molina	20221020102
Jan Henrik Sánchez Jerez	20231020130
Juan Carlos Quintero Rubiano	20232020172
Juan Diego Peralta Barbosa	20231020146

7 de septiembre de 2026

Índice

1. Introducción **2**

2. Descripción de la aplicación **2**

3. Métricas con Source Monitor **2**
3.1. Métricas manejadas por la herramienta 2
3.2. Resultados obtenidos 2
3.3. Representación visual de las métricas (Diagramas) 2
3.4. Interpretación profunda de resultados 3

4. Métricas de Interfaz de Usuario (Lighthouse) **4**

5. Métricas con SonarQube **4**
5.1. Interpretación de resultados (SonarQube) 5

6. Conclusiones **5**

1. Introducción

El presente informe detalla el análisis de métricas de software aplicado al proyecto GLUD Fried Chicken (GFC). El objetivo principal es evaluar la calidad estructural del código fuente del backend mediante herramientas de análisis estático, asegurando que el diseño cumple con los estándares de mantenibilidad, legibilidad y eficiencia requeridos en el taller.

2. Descripción de la aplicación

GLUD Fried Chicken (GFC) es una aplicación CRUD desarrollada en Spring Boot para gestionar las operaciones de un restaurante. El sistema se compone de entidades clave como `Cliente`, `Comida`, `Pedido`, e `Ingrediente`. El diseño incorpora decisiones orientadas a la seguridad, implementando JSON Web Tokens (JWT) y codificación PBKDF2 para las contraseñas. Además, cuenta con una lógica robusta para la gestión de inventarios y personalización de pedidos.

3. Métricas con Source Monitor

3.1. Métricas manejadas por la herramienta

Source Monitor evalúa la estructura interna del código fuente midiendo atributos clave como el volumen absoluto del sistema (líneas de código y sentencias), la complejidad ciclomática (que evalúa la cantidad de rutas independientes en la lógica de un método), y la profundidad de bloques, la cual mide el nivel de anidamiento de las estructuras de control.

3.2. Resultados obtenidos

A continuación, se presentan las métricas base del proyecto extraídas de Source Monitor:

Métrica	**Valor**	**Rango aceptable**
Líneas de Código (M0)	3,809	Depende del proyecto
Sentencias (M1)	2,277	Depende del proyecto
Métodos por Clase (M6)	5.37	< 10
Sentencias por Método (M7)	3.04	< 15
Complejidad Máxima (M10)	9	< 10
Profundidad Promedio (M13)	1.22	< 3

Cuadro 1: Métricas obtenidas con Source Monitor (Checkpoint: Baseline)

3.3. Representación visual de las métricas (Diagramas)

Para una mejor comprensión de la distribución de la complejidad y las métricas del proyecto, Source Monitor genera representaciones gráficas.

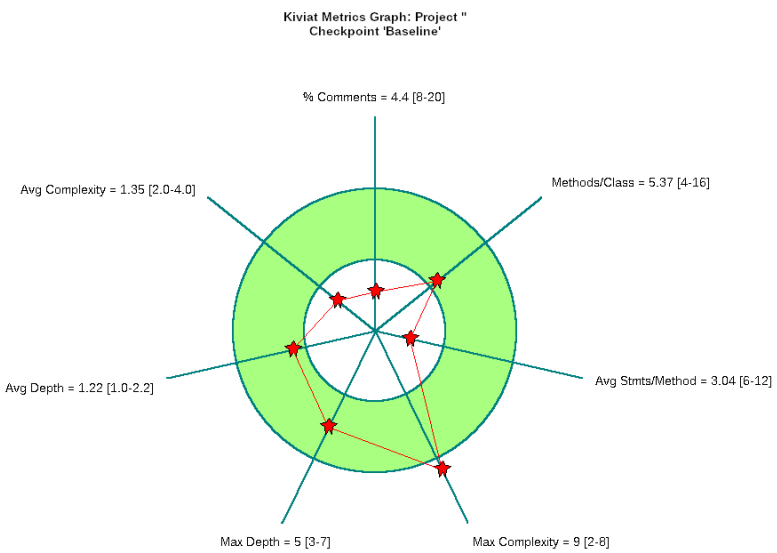


Figura 1: Diagrama Kiviat generado por Source Monitor

3.4. Interpretación profunda de resultados

Al examinar a fondo las métricas arrojadas por Source Monitor, se pueden identificar patrones arquitectónicos y de diseño muy sólidos dentro del backend del proyecto, estructurados en los siguientes frentes:

1. Modularidad y Responsabilidad Única:
El proyecto consta de 78 archivos y 62 clases/interfaces, sumando un total de 3,809 líneas de código. El promedio de **5.37 métodos por clase** y apenas **3.04 sentencias por método** demuestra una adherencia estricta al Principio de Responsabilidad Única (SRP). Los métodos son extremadamente cortos y concisos, lo que facilita enormemente la escritura de pruebas unitarias (como se evidencia en las clases de test registradas) y previene la existencia de "God Classes"(clases monolíticas). Adicionalmente, el alto número de sentencias de llamada a métodos (1,012 *Method Call Statements*) indica un sistema basado fuertemente en la delegación de tareas entre capas (Controladores, Servicios, Repositorios), típico y deseable en la arquitectura de Spring Boot.

2. Complejidad y Flujo de Control:
La complejidad promedio del proyecto es de tan solo **1.35**, respaldada por un bajo porcentaje de sentencias de ramificación (**3.6 % Branch Statements**). Esto significa que la ejecución del código es casi lineal en su mayor parte. No obstante, la herramienta identifica componentes críticos de negocio y seguridad donde la complejidad aumenta justificadamente:

- `Pbkdf2CodificadorContrasena.comprobar()`: Complejidad máxima de 9, natural dado el manejo de arreglos y algoritmos criptográficos.
- `IngredienteService.crear()`: Complejidad de 8 y 17 sentencias, concentrando validaciones de reglas de negocio al ingresar stock.
- `MovimientoInventario.aplicar()` y `JwtAuthenticationFilter.doFilterInternal()`:

Ambas con complejidad de 6. Representan el núcleo del control de existencias y el filtro de seguridad de peticiones HTTP, respectivamente.

3. Anidamiento y Estructura de Bloques:
La **profundidad de bloque máxima es 5**, presente únicamente en 9 sentencias de todo el proyecto (asociadas a validaciones complejas como `DetallePedido.cantidadesNecesarias()`). Sin embargo, la distribución general es sobresaliente: de las 2,277 sentencias, más de 2,150 se encuentran en profundidades de nivel 0, 1 y 2. Una profundidad promedio de **1.22** garantiza que los desarrolladores no tengan que hacer un seguimiento mental exhaustivo de estructuras condicionales anidadadas (ej. evitar el antipatrón de código pirámide o *Arrow Anti-Pattern*).

4. Documentación (Oportunidad de Mejora):
El único factor verdaderamente crítico arrojado por Source Monitor es el porcentaje de líneas con comentarios, el cual se ubica en **4.4 %**. Para aplicaciones de nivel empresarial, este indicador suele sugerirse por encima del 10 %. Aunque el código es altamente descriptivo (Çlean Code") debido a la baja cantidad de sentencias por método, se recomienda aumentar la documentación de las reglas de negocio, específicamente en las validaciones de dominio y JavaDocs.

4. Métricas de Interfaz de Usuario (Lighthouse)

Para complementar el análisis del backend, se evaluó el rendimiento y calidad del frontend (Çrujiente hasta el alma") utilizando Google Lighthouse. Es importante destacar que estas métricas fueron tomadas específicamente desde la **landing page** del aplicativo; este detalle es crucial ya que cada página web posee un *score* independiente basado en sus propios recursos, peso e interacciones DOM. Los resultados de la landing page destacan la eficiencia de la interfaz principal:

- **Performance (Rendimiento):** 99/100, con un First Contentful Paint de 0.5s.
- **Accesibilidad:** 95/100.
- **Mejores Prácticas:** 96/100.
- **SEO:** 100/100.

La vista inicial no presenta bloqueos en el hilo principal (Total Blocking Time de 0 ms), aunque la herramienta sugiere ligeras optimizaciones en el tamaño de las imágenes y ajustes menores en el contraste de colores para alcanzar la puntuación perfecta en accesibilidad.

5. Métricas con SonarQube

El análisis estático del backend fue complementado con el *Project Health Dashboard* de SonarQube para evaluar los atributos de calidad externos. El *Quality Gate* general del proyecto alcanzó el estado de **Passed** (cumpliendo exitosamente todas las condiciones), registrando un total de 32 *Open issues* y un índice de **0.0 %** en Duplicaciones de código.

A continuación, se presenta el desglose de los diferentes apartados de salud del proyecto:

Snapshot (Categoría)	Rating	Issues	Observaciones
Security (Seguridad)	A	0	1 resuelto exitosamente (MTTR: 13 min)
Security Hotspots	A	0	Sin vulnerabilidades críticas reportadas
Reliability (Fiabilidad)	A	5	Severidad: 20 % Medium, 80 % Info
Maintainability (Mantenibilidad)	A	31	Concentra el grueso de los *Open issues*

Cuadro 2: Desglose de Salud del Proyecto (SonarQube)

5.1. Interpretación de resultados (SonarQube)

Los resultados del *Project Health Dashboard* reafirman la alta calidad y robustez del backend desarrollado:

En el frente de **Seguridad (Security y Hotspots)**, el proyecto ostenta una calificación **A** perfecta con 0 vulnerabilidades abiertas. Además, destaca un sobresaliente *Mean Time To Repair (MTTR)* de apenas **13 minutos** en la resolución del único incidente introducido y cerrado recientemente, demostrando una alta capacidad de respuesta del equipo frente a brechas de seguridad.

En cuanto a la **Fiabilidad (Reliability)**, el análisis registra un Rating **A** con tan solo 5 issues. Es importante destacar que el desglose de severidad de estos problemas arroja que el **80 % son netamente informativos (Info)** y solo el **20 % son de nivel medio (Medium)**. No existe ningún fallo clasificado como *Blocker* o *Critical*, lo que garantiza la estabilidad en la ejecución del software.

Por el lado de la **Mantenibilidad (Maintainability)**, se identificaron 31 issues de código. Aunque representan la gran mayoría de los 32 problemas abiertos en general, mantener una calificación **A** indica que el esfuerzo y tiempo necesario para saldar esta deuda técnica es mínimo y no compromete el avance del proyecto a mediano plazo.

El único punto crítico revelado sigue siendo la **cobertura de pruebas (Coverage) situada en un 0.0 %**. Dado que el análisis interno demostró la existencia de múltiples métodos de pruebas, este 0 % confirma que la herramienta no está recibiendo correctamente el reporte de ejecución (ej. archivo de JaCoCo). Se requiere ajustar el pipeline o el plugin del scanner para que la cobertura real sea procesada.

6. Conclusiones

La evaluación integral de GLUD Fried Chicken demuestra un proyecto de alta calidad técnica y excelentes prácticas de ingeniería. Desde el backend, Source Monitor y SonarQube convergen en evidenciar un código modular, seguro (Rating A constante) y mantenible, con un 0 % de duplicación de código y un tiempo de resolución de vulnerabilidades (MTTR) sobresaliente de 13 minutos. Por su parte, el análisis de Lighthouse sobre la *landing page* garantiza una primera impresión inmejorable para el usuario, con un rendimiento web de 99/100. Las tareas clave para los siguientes sprints se reducen a mejorar la densidad de los comentarios internos y arreglar la vinculación del *Coverage* de pruebas unitarias hacia SonarQube.